PROGRAMMING

Competency 9: Develops algorithms to solve problems and uses python programming language to encode algorithms

Competency Level 9.1: Uses problem-solving process
Competency Level 9.2: Explores the top down and stepwise refinement methodologies in solving problems
Competency Level 9.3: Uses algorithmic approach to solve problems

Learning Outcomes:

- Describes the steps of problem solving process
- Uses stepwise refinement methodology to solve problems
- Draws structures charts to illustrate a solution for a system
- Describes algorithms briefly
- Identifies the standard symbols used to draw flow charts
- Draws flow charts to illustrate solutions to a given problem
- Writes pseudo codes to illustrate solutions to a given problem
- Uses hand traces to verify the solutions

Algorithm Development
An Algorithm is a finite sequence (a finite number of elements) of well-defined (clear and unambiguous) instructions, typically to solve a problem.

The problem solving process involves the steps depicted in the following figure in cyclic way.

The problem solving process is important not only in solving mathematical problems but also in real life problems.

Examples:

Switching on an air conditioner as the atmospheric temperature rises is solving a mathematical inequality depending on a physical papermaker temperature.

Finding a suitable course to follow is solving a mathematical comparison of cost and benefits of different courses.

For all the examples like the ones mentioned above, a computer program can be written based on an Algorithm.

Modularization (Decomposition)

To understand the term Algorithm, let us look at the problem-solving process in general. The problem-solving process can be itemizedas follows:

- Identifying the problem
- Identifying the subproblems (causes of the main problem)
- Finding (sub) solutions to subproblems
- Connecting the sub-solutions logically to solve the main problem

For example, take a simple day to day situation of making a cup of tea. The process involves taking the ingredients as input, the process of mixing the ingredients to produce the output of a cup of tea.

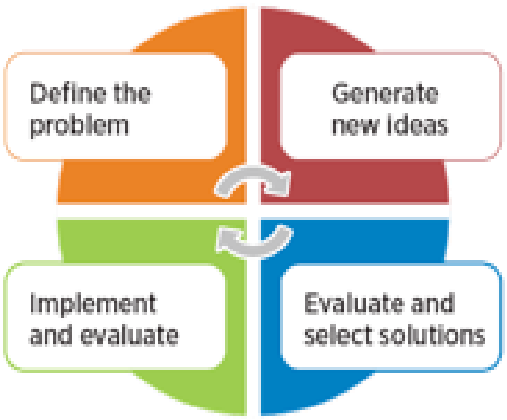
In some problems, it is necessary to break the big problem into small subproblems and consider the solution as a collection of (sub) solutions to different subproblems.

Example:
Main problem: inefficient administration in an organization
Sub Problem: Problems in financial division, problems in human resource division, problems in technical division, etc..

Note: This is what is done in the system design process.
The same methodology applies even in solving mathematical problems.

The above strategy for solving a problem is called Modularization or Decomposition Modularization means breaking a big problem into logical subproblems. In our examples, we broke or modularized the big problem into small problems that cannot be further broken logically. Inthe case of making a cup of tea, the modularization is boiling water, mixing the ingredients, etc.

In a real-world system design, in the first attempt, we may not get the best possible modularization. It is necessary to refine the initial model to make it better. In the refinement process, we may have to combine some subproblems or break into further subproblems to make the system more sensible. The process can be depicted graphically as follows.



Once the subproblems (or subsystems) are identified, they have to be documented in a visually comprehensible way. The best visually comprehensible way of writing is by using a figure.

Suppose that our problem is to check whether a person is eligible to vote at an election:

If the age is greater than or equal to 18, then eligible to vote
If the age is less than 18, then not eligible to vote

The following is the representation of the above problem and solution using what is known as a block diagram

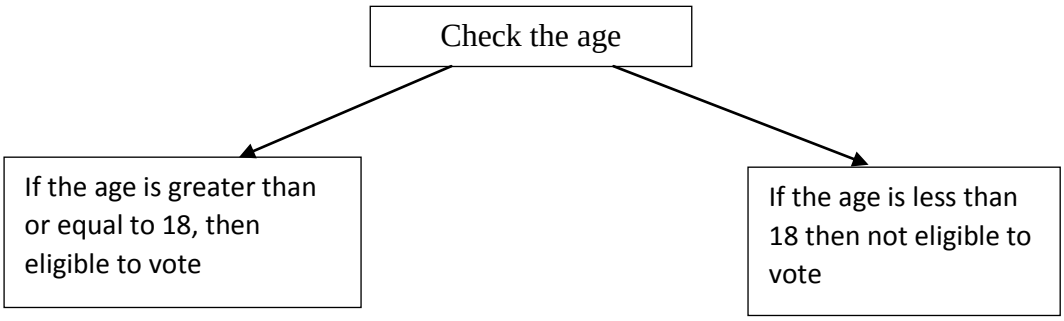


Figure No 2.1- Visually Comprehensible Representation

Algorithmic approach to solve problems

Algorithms can be represented as follows:

- Graphically : Flow chart
- Textually: Pseudocode

Flow chart / Pseudocode are converted into source code.

Graphically:

Once the system block diagram is done, it should be developed into a flow chart.Flow chart literally means the flow of actions to realize the components depicted in the block diagram.

- **Flow chart**

The flow chart uses the following standard symbol to describe a solution.

Start / End	
Input/ Output	
Decision Making	
Process (Action)	
Flow Lines	
Joining Paths (Connector)	
Subroutine	

Table 2.1 – Common Flowchart Symbols

Using the above symbols, we can draw the following flow chart to the problem discussed above.

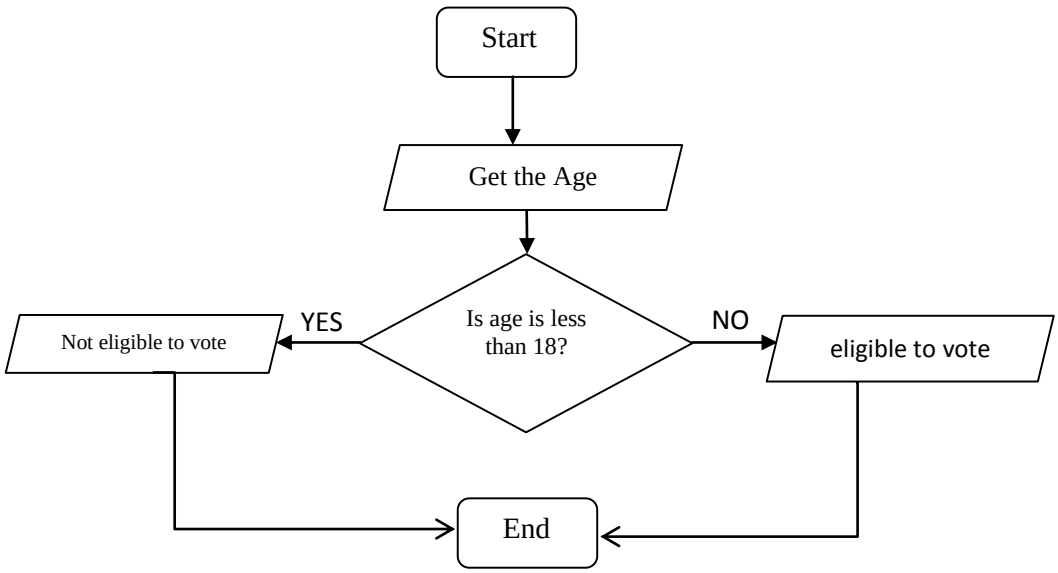


Figure 2.2: Flowchart for checking eligibility to vote

Textually:

- **Pseudocodes**

  The flow chart then needs to be represented as a Pseudocode. Pseudocode is written as a statement in English that is closer to computer programming languages but independent of any particular language.We do this because when the pseudocode is written, converting it into a suitable computer programming language is easy.

  The pseudocode for our example looks as below:

```
Begin
        Read age
        If age <18 then
                Display “not eligible to vote”
        Else
                Display “eligible to vote”
        Endif
End
```

- **Hand traces**

  Once the pseudocode is written, a manual check is done before converting it into a programming language. Thisis done by working with an appropriate example situation and line by line. In our example, we could enter or inputany age value and see what happens in the process. Thisis donein order tominimize possible errors when executing the program.

Competency Level 9.4: Compares and Contrasts different programming paradigms

Learning Outcomes:

- Describes the evolution of programming language in terms of generations
- Compares and contrasts imperative, declarative, object oriented languages

Evolution of programming languages

The programming language is a set of commands written in a specific format and grammar, and it has to be learnt just like learning another language to understand a foreign language to communicate with a foreigner. C, C++, Python, Pascal, Java, etc. are some common programming languages.

Once the pseudocode which is general and not specific to any programming language is written, it should be converted into a specific programming language of choice and based on the type of application and expertise.

The programming language is the language understandable to humans and to computers once compiled. There are many different languages that are suitable for different applications: scientific calculations, database management, business applications etc.

Programming paradigms

Computer programming languages can be categorized into two broad categories (depending on how they work), they are known as programming paradigms.

1. **Imperative languages**

This is the most common type. In this type, the problem is solved (the program is written) as a sequence of commands (imperatives).
Further, these languages can be categorized into three types:

- Procedural. e.g: C
- Object-oriented. e.g: Java, C++
- Parallel processing.e.g: Java

Declarative languages

This type is less common. In this type, there is no specific sequence and statements (declarations) can have a different flow.
Further, these languages can be categorized into three types:

- Logic. e.g: Prolog
- Functional data flow. e.g: Lisp
- Database. e.g: SQL

Competency Level 9.5: Explores the need of program translation and the type of program translators

Learning Outcomes:

- Describes the need of translation of a program
- Compares the source and object program
- Lists and briefly describes the types of program translators
- Briefly describes the function of linkers

Program translation and the type of program translators

Source Program and Object Program need to be translated into machine code.

Source code

A program written in a high-level language like C, Python, etc. can be understood by humans who know the particular language.

Object code

Source code which is not understandable by computers directly without being converted to what is known as the machine language, should be converted into machine language using an intermediate program. After converting into machine language, it is called as object code.

The machine language understandable by computers is made of digital commands in Boolean logic. This is something humans cannot learn and memorize due to the enormous amounts of digits and the vulnerability of mistakes.

To overcome the above-mentioned difficulty, once the human-understandable source code programming is done, an intermediate program converts it into a corresponding object code containing binary values.
There are two ways – interpreting and compiling - of conversion.

- **Interpreter**
  The interpreter converts the source code into an object code line by line. Due to its' nature, this process has to be repeated every time the program is run, and therefore, the execution time is high. Some languages like BASIC, FORTRAN, and Python use this type of conversion.

- **Compilers**
  A compiler converts the whole of the source code into object code at once. After the process, a permanent binary code known as object code is generated. Every time the program runs, the object code is executed and therefore, the execution time is low. Languages like Pascal, C use this approach. If a change is made to the source program, the compilation has to be done again, and new object code is generated.

- **Hybrid approach**
  When there are two or more ways of doing something, there are relative advantages and disadvantages, and to balance them, hybrid approaches are taken. An example is a hybridelectric car where there is a battery for town running, and an engine to run outside towns. Similarly, Interpreting and Compilation can be combined to improve the efficiency.